

TP Réseaux — Chat sur socket Unix

L'objectif de ce TP est de construire un **chat** en se basant sur les **sockets de Berkeley** (ou **sockets Unix**). Le chat sera installé sur le **réseau local** (restreint à la salle de classe).

La première version a une architecture de type **client-serveur**. Puis, la deuxième version permettra de transmettre des messages entre clients de manière **décentralisée**, c'est-à-dire sans passer par un serveur.

1. Démarrage

Étape 1 — Installation

Récupérer sur le serveur pédagogique les fichiers `client.c` et `parse_command.h`.

Compiler le fichier `client.c` avec la méthode de votre choix. Par exemple, exécuter dans un terminal :

```
cc -o client client.c
```

Exécuter le binaire dans un terminal :

```
./client
```

La commande `/q` permet de quitter le programme.

2. Client du chat général

Le premier objectif est de créer un client qui peut communiquer sur le **chat général**. Le chat est géré par le serveur. Le client se connecte, puis il dispose d'un **socket** pour envoyer des messages et réceptionner les messages transmis par le serveur.

Étape 2 — Connexion au serveur

L'adresse IP du serveur sur le réseau local est **169.254.61.46**. Il attend une connexion (TCP) sur le port **4242**. Établir une **connexion** avec le serveur.

Utiliser Wireshark pour vérifier que la connexion est établie : après les trois paquets TCP de connexion, le serveur vous envoie un message de bienvenue.

Indice

La connexion doit être établie **avant** la boucle principale.

Étape 3 — Réception de messages du chat général

Dès qu'un client transmet un message au serveur, ce dernier le transmet à tous les clients connectés (broadcast).

Dans la boucle principale, **réceptionner** les messages transmis par le serveur et les **afficher** sur votre terminal.

Indice

C'est le **thread parent** qui effectuera cette tâche.

Étape 4 — Envoi de messages sur le chat général

La fonction `scanf()` s'occupe de lire le contenu du terminal jusqu'à la première occurrence de `\n` (qui correspond à un appui sur la touche entrée).

Dans la boucle principale, **envoyer** au serveur le message écrit dans le terminal.

Indice

C'est le **thread enfant** qui effectuera cette tâche.

Étape 5 — Déconnexion du serveur

Lorsque vous tapez la commande `/q` dans le terminal, votre client doit initier la déconnexion au serveur, fermer la socket et terminer l'application.

Ajouter la phase de **déconnexion** avec le serveur.

Indice

Attention : chaque thread a sa propre valeur de la variable `quit`.

3. Message privé

L'objectif suivant est d'ajouter une nouvelle fonctionnalité à notre chat : les messages privés.

Nous adaptons une **approche décentralisée** : les messages privés seront transmis en direct entre les clients, sans intervention du serveur. Pour faire simple, les messages privés seront transmis **sans connexion** (UDP).

Étape 6 — Créer une socket d'écoute

Pour transférer les messages privés sans passer par le serveur, il va falloir ouvrir un **port d'écoute**. **Créer** une socket en mode **sans connexion** (UDP) et l'**attacher** au port 5555.

Indice

C'est une socket d'écoute qui accepte des messages depuis n'importe quelle adresse, sur son port 5555.

On ne va pas utiliser cette socket tout de suite. On commence par s'occuper de l'**envoi** de messages privés.

Étape 7 — Une nouvelle commande pour les messages privés

Pour transmettre des messages privés dans le chat, on utilisera la commande `/mp [IP] [MESSAGE]`. Par exemple :

```
/mp 169.254.25.1 Ceci est un message privé
```

Pour détecter l'utilisation de cette commande, la fonction `parse_mp` est fournie dans le fichier `parse_command.h`. Utiliser cette fonction pour **détecter l'utilisation de la commande de messages privés**. Afficher les valeurs d'adresse IP et de message détectées.

Indice

La fonction `int parse_mp(char* src, char* ip_addr, char* msg)` prend en arguments :

- `src` : la chaîne de caractères rentrée par l'utilisateur dans le prompt
- `ip_addr` : une chaîne de caractères qui recevra l'adresse IP (si la commande `/mp` a été utilisée)
- `msg` : une chaîne de caractères qui recevra le contenu du message (si la commande `/mp` a été utilisée)

Elle renvoie les valeurs :

- `-1` : si elle échoue
- `0` : si la commande `/mp` n'a pas été utilisée
- `1` : si la commande `/mp` a été utilisée

Étape 8 — Envoyer des messages privés

Si la commande `/mp` est utilisée, transmettre le message à l'adresse IP spécifiée, sur son port 5555. Implémenter l'**envoi de messages en direct**, sans connexion (UDP).

Indice

Il faudra créer une socket temporaire pour effectuer l'envoi. Et oui encore une !

La dernière étape est la réception des messages privés sur notre port 5555. Il va falloir **scruter** la socket qui est attachée à ce port. Nous utiliserons un **thread** supplémentaire.

Étape 9 — Créer un thread supplémentaire

Créer un **thread supplémentaire** en ré-utilisant la fonction `fork()` dans le thread parent.

ATTENTION : Penser à bien fermer le nouveau thread avant la terminaison du programme.

Indice

Dans le code suivant, quelles sont les valeurs de `pid[0]` et `pid[1]` dans le thread parent ? dans le premier thread enfant ? et le deuxième thread enfant ?

```
int pid[2] = {0, 0};
pid[0] = fork();
if(pid[0] != 0) { // thread parent
    pid[1] = fork();
}
```

Étape 10 — Recevoir des messages privés

Utiliser un des deux threads enfants pour implémenter la **réception de messages privés** sur la socket attachée au port 5555.

Indice

Pour tester le bon fonctionnement des messages privés, créer un deuxième client qui utilise un port différent (par exemple 5556) et envoyer un message privé à l'adresse 127.0.0.1 (votre adresse de bouclage locale).